

Marco de trabajo para mejorar la calidad de requerimientos ágiles: una primera aproximación

Resumen

En el marco de la Ingeniería de Software Empírica, este trabajo presenta un marco de trabajo para la gestión de requerimientos en entornos ágiles, con el objetivo de mejorar la claridad, completitud, consistencia y trazabilidad de los mismos desde etapas tempranas del ciclo de vida del desarrollo. A partir de una revisión sistemática de la literatura y una encuesta a profesionales del sector, se identificaron desafíos recurrentes en la definición y documentación de los requerimientos ágiles. Como respuesta, se propone un modelo estructurado que integra criterios cuantificables y métricas operativas para evaluar la calidad de los requerimientos, así como una primera aproximación de herramienta que mediante un kernel que orqueste los módulos de clasificación, validación y priorización. El enfoque propuesto ofrece una base sólida para la mejora continua en la gestión de requerimientos ágiles, al tiempo que abre la puerta a futuras implementaciones automatizadas. La validación empírica del modelo y el desarrollo de herramientas de soporte serán los pasos siguientes para consolidar este marco y evaluar su impacto en la calidad de los productos de software.

1. Introducción

El Desarrollo Ágil de Software (ASD, por sus siglas en inglés *Agile Software Development*) ha sido ampliamente adoptado en la industria debido a su flexibilidad, su carácter iterativo y su énfasis en la entrega temprana de valor. Sin embargo, a pesar de su popularidad, la gestión de requerimientos en estos entornos continúa presentando desafíos significativos, particularmente en áreas críticas como la completitud, trazabilidad y priorización de los requerimientos [1, 2]. En numerosos casos, las organizaciones implementan prácticas ágiles de manera intuitiva o parcial, sin realizar una evaluación sistemática de su impacto en la calidad tanto del proceso como del producto [3].

Estudios empíricos han identificado que la definición deficiente de requerimientos constituye una de las principales causas de inestabilidad y demoras en proyectos ágiles [4]. No obstante, los enfoques tradicionales de Ingeniería de Requerimientos (RE, por sus siglas en inglés *Requirements Engineering*) suelen presentar limitaciones para abordar los desafíos específicos de estos entornos, lo que refuerza la necesidad de investigar y validar, mediante métodos empíricos, aquellas prácticas que resultan más efectivas en contextos ágiles [5]. En este sentido, la Ingeniería de Software Empírica (ESE, por sus siglas en inglés *Empirical Software Engineering*) adquiere especial relevancia, ya que permite generar evidencia a partir de experiencias reales de equipos de desarrollo y así evaluar cómo distintas estrategias de gestión de requerimientos inciden en la calidad del proceso y del producto [6, 7].

En este marco, la ESE se consolida como un enfoque idóneo para abordar los desafíos complejos del desarrollo de software, especialmente en contextos ágiles, donde la dinámica de cambio exige soluciones flexibles y basadas en evidencia. Bajo esta perspectiva, en una primera etapa se implementaron diversas metodologías empíricas, iniciando con una Revisión Sistemática de la Literatura (RSL), con el fin de proporcionar una comprensión integral sobre las estrategias y prácticas de gestión de requerimientos que podrían contribuir a mejorar la calidad de los procesos ágiles. Los hallazgos de esta RSL revelaron áreas críticas que impactan negativamente en la calidad del software, como la incompletitud de los requerimientos, su baja trazabilidad y las dificultades para priorizarlos de manera efectiva [8].

A partir de estos hallazgos, se diseñó una encuesta con el objetivo de relevar el estado actual de la práctica de gestión de requerimientos en entornos reales de ASD. Esta encuesta no solo buscó reflejar las prácticas prevalentes en equipos de desarrollo de software, sino también validar y expandir el conocimiento generado por la RSL, proporcionando datos empíricos que podrían orientar mejoras prácticas en la industria. La encuesta se centró en relevar información sobre aspectos clave de la gestión de requerimientos, tales como la adopción de enfoques ágiles, la captura y priorización de

requerimientos, las técnicas de documentación y la gestión de cambios, identificados previamente como problemáticas recurrentes con impacto potencial en la calidad de los procesos de desarrollo de software.

Los resultados de la encuesta mostraron patrones consistentes con los hallazgos de la RSL, evidenciando nuevamente la presencia de deficiencias en la completitud, calidad, trazabilidad y priorización de los requerimientos. Estos hallazgos subrayan la necesidad de nuevas estrategias que puedan acompañar a los equipos de desarrollo desde las etapas iniciales de los proyectos. Mejorar la calidad y la relevancia contextual de los procesos ágiles requerirá un enfoque más integrado en la captura, análisis y evolución de los requerimientos a lo largo del ciclo de vida del desarrollo. Por lo tanto, como parte de las futuras fases de esta investigación, se propone el diseño de nuevos enfoques capaces de abordar los impedimentos identificados mediante la automatización y agilización de aspectos clave de la gestión de requerimientos.

En esta segunda etapa, el estudio se centra en el diseño de un marco de trabajo integral destinado a asistir la gestión de requerimientos en entornos ágiles. Este marco se compone de dos elementos fundamentales: un modelo conceptual de calidad para el análisis, calificación y clasificación de requerimientos, y una herramienta de software que automatiza los componentes clave del modelo. La propuesta se apoya en los hallazgos empíricos previos y establece tanto los fundamentos teóricos como los principios de automatización necesarios para su implementación práctica.

El modelo de calidad considera indicadores tanto cuantitativos como cualitativos para evaluar atributos clave de los requerimientos, tales como claridad, completitud, consistencia, modularidad, entre otros; mientras que la herramienta automatizada integra mecanismos de evaluación sistemática, retroalimentación inteligente y enriquecimiento textual. Este enfoque no busca reemplazar el juicio experto, sino complementarlo mediante una evaluación sistemática y adaptable, orientada a mejorar la calidad de los requerimientos y, con ello, mejorar tanto los procesos de desarrollo como los productos resultantes.

La motivación central de este trabajo es presentar una primera aproximación de un marco de trabajo compuesto por un modelo de evaluación de calidad y una herramienta que automatice los componentes de dicho modelo. La propuesta ha sido ideada como fundamento para la clasificación, evaluación y refinamiento de requerimientos ágiles de software, proporcionando un marco que favorezca su gestión eficiente y consistente.

Este trabajo se enmarca dentro de la línea de investigación [REDACTED], desarrollada en [REDACTED].

[REDACTED] Esta línea de investigación se propone articular la investigación empírica con desarrollos tecnológicos concretos que aporten valor práctico a la industria del software, generando al mismo tiempo conocimiento transferible a contextos educativos y profesionales.

El resto del trabajo se estructura de la siguiente manera: la sección 2 presenta las motivaciones y el marco empírico que sustentan la propuesta; la sección 3 describe diseño y estructura de una primera aproximación a modelo de calidad; la sección 4 expone una propuesta preliminar de automatización de componentes del modelo; y la sección 5 plantea las conclusiones y líneas de trabajo futuro.

2. Motivaciones

La creciente complejidad de los proyectos de software, junto con la adopción generalizada de enfoques ágiles, ha evidenciado la necesidad de contar con mecanismos más robustos para gestionar eficazmente los requerimientos desde las primeras fases del desarrollo. En este contexto, como parte de una etapa previa de esta línea de investigación, se llevó a cabo una Revisión Sistemática de la Literatura (RSL) y una encuesta a profesionales de la industria, cuyos resultados fueron presentados en CONAIIISI 2024 [9]. Estos estudios permitieron construir un panorama integral de las prácticas actuales, los obstáculos frecuentes y las oportunidades de mejora en la gestión de requerimientos en contextos ágiles.

La RSL analizó 98 estudios relevantes, de los cuales 42 cumplieron con criterios de calidad y fueron clasificados como estudios primarios. Entre los hallazgos más significativos se destacan las dificultades para lograr requerimientos completos, trazables y consistentes dentro de marcos ágiles de trabajo. Además, se identificó la falta de integración entre prácticas tradicionales de ingeniería de requerimientos y enfoques ágiles, así como la escasa orientación a la mejora continua de los requerimientos a lo largo del proceso.

La encuesta, aplicada a 40 profesionales de la industria del desarrollo de software, aportó información empírica complementaria. Los resultados mostraron que el 87,5% de los encuestados utilizaba Scrum, y que las técnicas predominantes de captura de requerimientos eran entrevistas con usuarios y el uso de historias de usuario. No obstante, el 72,5% de los participantes indicó dificultades persistentes con la incompletitud de los requerimientos, y más del 60% señaló problemas para gestionar interdependencias y mantener documentación actualizada. Además, los requerimientos no funcionales mostraron una baja atención sistemática, y se identificaron impedimentos como la escasa participación

de los stakeholders y la falta de herramientas automatizadas de apoyo.

Estos resultados revelan una necesidad crítica: la de contar con modelos estructurados que no solo permitan evaluar la calidad de los requerimientos existentes, sino también proporcionar guías y mecanismos concretos para su mejora. En particular, se observa una carencia de enfoques que integren valoraciones objetivas y estrategias de enriquecimiento adaptadas a contextos ágiles.

A partir de este diagnóstico, la motivación central de este trabajo es desarrollar una primera aproximación a un marco de trabajo compuesto por un modelo de evaluación de calidad y una herramienta que automatice los componentes de dicho modelo. La propuesta busca sentar las bases para la clasificación, evaluación y refinamiento de requerimientos ágiles de software, ofreciendo un marco que favorezca su gestión eficiente y consistente.

Por una parte, el objetivo del modelo es formalizar criterios de evaluación y establecer estructuras reutilizables que puedan ser integradas en futuras herramientas inteligentes. Y, por otra parte, la finalidad de la herramienta es permitir la gestión automática de los componentes del modelo ofreciendo información sobre el nivel de calidad de los requerimientos ágiles que se procesen.

Con esta iniciativa se busca dar continuidad a la línea de investigación previa, enfocándose ahora en el diseño conceptual y metodológico necesario para fundamentar desarrollos posteriores que incluyan agentes inteligentes o sistemas de recomendación automatizados.

La siguiente sección describe la primera aproximación al modelo propuesto, diseñado para responder a estas necesidades.

3. Primera aproximación al modelo de calidad de gestión de requerimientos ágiles

Un modelo de calidad en Ingeniería de Software constituye un marco conceptual y metodológico que establece criterios, métricas y procesos sistemáticos para evaluar, medir y mejorar la calidad de productos o procesos de software [10]. Estos modelos proporcionan una estructura formal que permite identificar características deseables, definir indicadores cuantificables y establecer procedimientos reproducibles para la evaluación de la calidad [11]. En el contexto específico de la gestión de requerimientos, un modelo de calidad se orienta hacia la evaluación de aspectos como claridad, completitud, consistencia, verificabilidad y trazabilidad de las especificaciones [12].

El desarrollo de un modelo de calidad específico para requerimientos ágiles responde a las particularidades que

estos presentan en comparación con los tradicionales. Al expresarse con frecuencia en forma de historias de usuario, los requerimientos ágiles priorizan la comunicación directa, la flexibilidad y la evolución incremental por sobre la documentación exhaustiva [13]. Esta naturaleza dinámica y menos formal introduce desafíos únicos en la evaluación de calidad, dado que los criterios convencionales resultan insuficientes o demasiado rígidos para este contexto [14].

La implementación de un modelo de calidad para requerimientos ágiles cumple múltiples propósitos fundamentales. Primero, proporciona un mecanismo sistemático para identificar y diagnosticar deficiencias en la especificación de requerimientos antes de que estas se propaguen a etapas posteriores del desarrollo, reduciendo así el costo de corrección [15]. Segundo, facilita la estandarización de criterios de calidad entre equipos de desarrollo, promoviendo la consistencia en la evaluación independientemente del evaluador humano. Tercero, permite la automatización de procesos de revisión y refinamiento, liberando recursos humanos para actividades de mayor valor agregado [16].

Además, un modelo de calidad bien estructurado contribuye a la mejora continua del proceso de especificación de requerimientos, proporcionando retroalimentación sistemática que permite a los equipos identificar patrones recurrentes de deficiencias y desarrollar estrategias preventivas [17]. En el contexto de metodologías ágiles, donde la velocidad de desarrollo es crítica, la disponibilidad de mecanismos automatizados de evaluación de calidad se convierte en un factor diferenciador que puede impactar significativamente en la productividad del equipo.

El modelo propuesto en este trabajo se estructura en componentes interrelacionados que conforman un flujo de proceso integral para la evaluación de requerimientos ágiles. En las subsecciones siguientes se presentarán detalladamente cada uno de estos componentes, así como las relaciones y dependencias entre ellos que dan lugar al flujo del proceso de evaluación. Esta arquitectura modular permite tanto la comprensión conceptual del modelo como su implementación práctica en herramientas automatizadas.

3.1. Componentes del modelo

El modelo conceptual ha sido diseñado para que los requerimientos, en su forma original, sean analizados, clasificados, reestructurados y organizados antes de transformarse en historias de usuario. De este modo, se busca mejorar la calidad y trazabilidad de los requerimientos, asegurando que los equipos dispongan desde el inicio de insumos claros, completos y priorizados.

La propuesta se fundamenta en principios establecidos por normas internacionales como ISO/IEC 25010 (modelo de calidad del software) [18], CMMI

(modelo de madurez de procesos) [19] y QuAM (modelo de calidad ágil) [20, 21], adaptados a las características y necesidades específicas de los entornos ágiles.

El modelo consta de tres componentes fundamentales: clasificación, validación y priorización de requerimientos. Se propone un flujo metodológico desde la recolección hasta la consolidación de los requerimientos, empleando una plantilla estandarizada que permite documentar los resultados de manera uniforme y consistente. Para mayor detalle, se exponen a continuación cada uno de los 3 componentes del modelo.

3.1.1. Clasificación de requerimientos

Una correcta clasificación de los requerimientos es el primer paso para lograr una gestión eficiente del desarrollo de software. Clasificar significa entender en profundidad la esencia de cada requerimiento, su objetivo, sus implicancias técnicas y su relación con el valor de negocio. La clasificación no solo permite una organización más ordenada del backlog, sino que también habilita procesos de análisis, trazabilidad y toma de decisiones más precisos en las etapas de diseño, implementación y mantenimiento. Este componente del modelo se fundamenta en el estándar ISO/IEC 25010, que establece una noción integral de calidad del software incluyendo atributos funcionales y no funcionales. A partir de este enfoque, el modelo distingue los siguientes tipos de requerimientos:

- **Requerimientos funcionales:** se refieren a las funcionalidades que el sistema deberá proporcionar. Estos requerimientos representan acciones concretas que el sistema debe ser capaz de ejecutar y que están directamente relacionados con el comportamiento esperado del producto frente a diferentes entradas o situaciones (por ejemplo, *"el sistema permitirá a los usuarios autenticar su identidad mediante contraseña y segundo factor"*). Según ISO/IEC 25010, los requerimientos funcionales están directamente relacionados con la característica de funcionalidad y su subcaracterística de adecuación funcional.
- **Requerimientos no funcionales:** describen los atributos de calidad que debe cumplir el sistema, como rendimiento, seguridad, portabilidad, compatibilidad, mantenibilidad o usabilidad. Aunque no definen qué hace el sistema, establecen cómo debe hacerlo (por ejemplo, *"el sistema deberá ser capaz de procesar 1000 solicitudes concurrentes sin degradar el tiempo de respuesta por debajo de 2 segundos"*). Estos requerimientos están alineados con la mayor parte de las características del modelo de calidad de ISO/IEC 25010.
- **Requerimientos técnicos:** agrupan condiciones específicas necesarias para la implementación del sistema, como entornos de ejecución, herramientas requeridas, restricciones de lenguaje o tecnologías

impuestas por la infraestructura. Estos requerimientos pueden derivarse tanto de necesidades funcionales como no funcionales, y son clave para el diseño arquitectónico (por ejemplo, *"el sistema deberá integrarse mediante API RESTful con el sistema de pagos externo"*).

- **Requerimientos de negocio:** representan las necesidades y objetivos de alto nivel del cliente o de la organización. Suelen originarse a partir de expectativas estratégicas y se traducen posteriormente en requerimientos funcionales y no funcionales (por ejemplo, *"el sistema deberá permitir reducir el tiempo promedio de atención al cliente en un 30%"*). Su adecuada identificación permite alinear el desarrollo con la misión y visión del negocio.

Junto con la clasificación por tipo, el modelo considera una clasificación complementaria según tres dimensiones fundamentales. En primer lugar, la **complejidad** estima el esfuerzo necesario para implementar el requerimiento, considerando aspectos técnicos, organizacionales y de integración. Los requerimientos de alta complejidad requieren planificación detallada, revisiones cruzadas y más recursos para su ejecución, y clasificar la complejidad ayuda a anticipar riesgos y gestionar expectativas.

En segundo término, las **dependencias** analizan las relaciones que un requerimiento mantiene con otros dentro del sistema. Pueden ser requerimientos que se deben implementar previamente, componentes que interactúan directamente o condiciones externas que afectan su funcionamiento, y detectar estas dependencias tempranamente es vital para evitar cuellos de botella, asegurar la coherencia funcional y reducir retrabajos.

Finalmente, el **impacto** evalúa cuán relevante es el requerimiento en relación con los objetivos del proyecto o del cliente. Un requerimiento de alto impacto puede ser determinante para el éxito o fracaso del producto, y esta dimensión permite establecer jerarquías y tomar decisiones de prioridad fundamentadas.

Para apoyar la clasificación y validación de los requerimientos, se utiliza una matriz de clasificación basada en el modelo CMMI. Esta herramienta facilita el registro de las características clave de cada requerimiento, tales como el tipo, la complejidad, las dependencias y el impacto, permitiendo realizar un análisis cruzado de los datos, tal como se muestra en la Tabla 1. Además, permite detectar combinaciones críticas (por ejemplo, requerimientos técnicos de alta complejidad y alto impacto con muchas dependencias), facilitando la toma de decisiones en etapas posteriores como la validación, la asignación de recursos o la planificación de iteraciones. Además, brinda una base estructurada para futuras automatizaciones o integraciones con sistemas de inteligencia artificial.

Tabla 1. Matriz de clasificación de requerimientos

ID Req	Descripción breve	Tipo	Complejidad	Dependencias	Impacto	Estado actual	Observaciones
RQ 01	Registro de usuario	Funcional	Media	RQ03 (Base de datos)	Alto	En análisis	Depende de diseño de modelo de datos
RQ 02	Envío de notificaciones	No funcional	Alta	RQ05 (Configuración SMTP)	Medio/Alto	Pendiente	Evaluar impacto en rendimiento
RQ 03	Crear base de datos de usuarios	Técnica / Soporte	Alta	—	Crítico	Aprobado	Crítico, habilita RQ01 y RQ06
RQ 04	Validación de datos en frontend	Funcional	Baja	RQ01	Medio	En desarrollo	Independiente de backend
RQ 05	Configurar servidor de correos	Técnica / Infra	Media	—	Medio	En análisis	Necesaria para notificaciones (RQ02)
RQ 06	Login con verificación en dos pasos	Funcional	Alta	RQ01, RQ03	Alto	Pendiente	Múltiples dependencias, priorizar

3.1.2. Validación de requerimientos

Una vez clasificados, los requerimientos deben ser validados para garantizar que representan fielmente las necesidades del cliente, son técnicamente viables y están correctamente formulados. Esta etapa es clave para asegurar que los requerimientos no solo estén bien escritos, sino que también sean útiles, coherentes, realizables y relevantes dentro del contexto del proyecto. Validar no implica únicamente verificar su presencia en un documento, sino confirmar su calidad sustantiva y su alineación con los objetivos del negocio y las capacidades técnicas del equipo.

El proceso de validación toma como base los lineamientos de la norma IEEE 830 (especificación de requerimientos de software) [22] y las prácticas de modelos como QuAM, incorporando criterios explícitos que permitan revisar sistemáticamente cada requerimiento antes de ser incluido en el backlog. La validación se convierte así en una instancia de control de calidad del conocimiento recolectado durante la etapa de relevamiento, donde cada criterio de calidad se evalúa mediante métricas específicas que proporcionan una base objetiva y medible para la toma de decisiones.

El modelo define los siguientes criterios de validación, asociados a métricas que permiten su evaluación cuantitativa:

- **Necesario:** el requerimiento debe ser esencial para el propósito del sistema, evaluando su alineación con los objetivos de negocio y su impacto en la funcionalidad del sistema.
- **Apropiado:** debe presentarse con el nivel de detalle y formato adecuados para la etapa de desarrollo, medido mediante índices de completitud estructural y adherencia a plantillas.
- **No ambiguo:** su redacción debe ser precisa y sin múltiples interpretaciones, evaluada cuantificando la presencia de términos imprecisos o construcciones sintácticas problemáticas.
- **Completo:** debe cubrir la necesidad en su totalidad,

comprobado mediante la presencia de elementos obligatorios y la cobertura de casos de uso.

- **Singular:** debe describir una única funcionalidad, medido por la cantidad de responsabilidades identificadas y la cohesión semántica del enunciado.
- **Factible:** debe ser implementable con los recursos y tecnologías disponibles, considerando la complejidad técnica frente a las capacidades del proyecto.
- **Verificable:** debe permitir demostrar su cumplimiento a través de pruebas, lo cual se mide por la presencia de criterios de aceptación testeables y condiciones observables.
- **Correcto:** debe reflejar la necesidad real del usuario, validado mediante trazabilidad hacia fuentes de información y confirmación con stakeholders.
- **Claro:** debe cumplir con estándares y convenciones de la organización, evaluado a partir de la legibilidad, el uso de terminología estándar y la estructura sintáctica.
- **Consistente:** no debe contradecir a otros requerimientos del conjunto, lo que se asegura mediante análisis de conflictos semánticos, contradicciones lógicas y solapamientos funcionales.
- **Viable:** debe poder implementarse dentro de los parámetros del proyecto, lo que se cuantifica analizando la factibilidad técnica, disponibilidad de recursos y plazos establecidos.

El modelo basa su evaluación en la observación de atributos específicos, que son medidos mediante un conjunto de métricas asociadas a cada criterio de calidad. Estas métricas permiten calcular un índice de calidad global para cada requerimiento y actúan como instrumentos de evaluación sistemática, aplicables tanto de forma automatizada como en sesiones colaborativas con Product Owners (POs), usuarios clave y miembros del equipo técnico. De este modo, se asegura una visión multifacética del requerimiento respaldada por evidencia medible.

3.1.3. Priorización de requerimientos

Una vez que los requerimientos han sido clasificados y validados, el siguiente paso es asignarles una prioridad. La priorización tiene como propósito garantizar que el esfuerzo del equipo se concentre en aquello que genera mayor valor para el negocio, satisface necesidades críticas del cliente o habilita funcionalidades clave para el sistema.

El modelo propone combinar criterios fundamentales con técnicas de priorización reconocidas, entre las que se incluyen MoSCoW [23, 24] y Planning Poker [25]. De esta manera, los criterios proporcionan la base de decisión, mientras que las técnicas ofrecen un marco práctico para estructurar la discusión y alcanzar consensos dentro del equipo.

Los criterios fundamentales para la priorización de los requerimientos son:

- **Valor de negocio**, que representa el impacto directo de un requerimiento en los objetivos estratégicos del cliente y la organización.
- **Esfuerzo estimado**, que hace referencia a los recursos (tiempo, personal, tecnología, etc.) necesarios para la implementación; un requerimiento con alto esfuerzo puede requerir más tiempo y recursos, lo que influye en su posición en la priorización.
- **Urgencia**, que determina la necesidad inmediata de implementación de un requerimiento en comparación con aquellos que pueden ser abordados en fases posteriores del proyecto.

Planning Poker permite consensuar de manera colaborativa el esfuerzo estimado de cada requerimiento, mediante la asignación de valores relativos por parte de los miembros del equipo.

MoSCoW clasifica los requerimientos como:

- Must Have (imprescindibles)
- Should Have (importantes pero no críticos)
- Could Have (convenientes si hay tiempo)
- Won't Have (no incluidos en la versión actual)

La integración de criterios y técnicas permite establecer prioridades de forma transparente, equilibrando la perspectiva estratégica del negocio con la capacidad operativa del equipo de desarrollo.

3.2. Funcionamiento del proceso del modelo

El modelo se operacionaliza a través de un flujo secuencial, iterativo y colaborativo, diseñado para integrarse en cualquier framework ágil (Scrum, Kanban, XP). Este flujo organiza de manera práctica los componentes definidos en la sección 3.1 —clasificación,

validación y priorización— y los articula en un proceso unificado que facilita su aplicación en contextos reales de desarrollo.

El proceso se estructura en cinco etapas principales: (i) identificación y clasificación, (ii) medición, (iii) validación colaborativa, (iv) priorización y consolidación, y (v) generación del resultado de análisis de requerimientos. Cada etapa toma como base los lineamientos conceptuales ya definidos y los convierte en pasos concretos que pueden repetirse de manera cíclica a lo largo del proyecto.

3.2.1. Identificación y Clasificación

El proceso se inicia con la recolección de requerimientos mediante entrevistas con usuarios, talleres de co-creación, revisión de documentación previa y análisis de procesos. En esta fase se busca capturar la mayor cantidad posible de información relevante, garantizando que los requerimientos reflejen tanto las necesidades funcionales como los objetivos estratégicos del negocio.

Una vez recolectados, los requerimientos se registran en bruto y luego se clasifican siguiendo los lineamientos definidos en la sección 3.1.1. Para ello se utiliza la matriz de clasificación presentada en la Tabla 1, que organiza cada requerimiento según su tipo (funcional, no funcional, técnico o de negocio) y de acuerdo con tres dimensiones complementarias: impacto, complejidad, dependencias e impacto.

Este procedimiento asegura que, desde el inicio, cada requerimiento cuente con un registro estructurado que facilite el análisis posterior y sienta las bases para su validación y priorización.

3.2.2. Medición

En esta etapa se evalúan las métricas asociadas a los criterios de validación definidos en la sección 3.1.2, con el fin de obtener una valoración cuantitativa y objetiva de la calidad de cada requerimiento.

Se proponen métricas de calidad que amplían la cobertura del cuarto componente del modelo QuAM para cada característica evaluada del requerimiento. La Tabla 2 ilustra esta relación, mostrando ejemplos de atributos positivos y negativos junto con los criterios de medición correspondientes.

Este enfoque asegura que la evaluación no se limite a observaciones cualitativas, sino que incorpore evidencia medible que facilite comparaciones y ajustes en etapas posteriores del proceso.

Tabla 2. Relación entre atributos y criterios de medición

Atributo Positivo		Atributo Negativo	
A4.1 Valor a la claridad de requerimientos		A.4.2 Valor a la ambigüedad de requerimientos	
Criterios de medición para A4.1	Valor	Criterios de medición para A4.2	Valor
El requerimiento está redactado con términos claros, estandarizados, y con interpretación unívoca entre stakeholders.	4	El requerimiento contiene múltiples términos ambiguos o subjetivos, generando contradicciones importantes entre stakeholders.	-4
La mayoría de los términos son claros, aunque existe alguna posible interpretación secundaria menor.	2	El requerimiento admite diferencias de interpretación menores que generan dudas puntuales.	-2
El requerimiento es comprensible en lo esencial, pero requiere revisión de glosario o ajustes menores de redacción.	1	El requerimiento presenta formulaciones confusas en algún punto; se detectan contradicciones en reuniones de revisión.	-1
No se detecta claridad destacada; el requerimiento cumple de manera básica con su función comunicativa.	0	No se encuentran conflictos de interpretación ni expresiones particularmente problemáticas.	0

3.2.3. Validación Colaborativa

En esta etapa se promueve la participación activa de los actores clave para asegurar la calidad de los requerimientos desde múltiples perspectivas. Se programan sesiones con los stakeholders relevantes, en las cuales se evalúa cada requerimiento aplicando las métricas asociadas a los criterios de validación definidos en la sección 3.1.2.

Como resultado de estas reuniones, se documentan las correcciones necesarias y se ajustan los requerimientos antes de que puedan avanzar a la siguiente fase del proceso.

De esta manera, la validación colaborativa refuerza la calidad de los requerimientos mediante una visión integral, que combina la evidencia cuantitativa con la experiencia y el juicio de los principales interesados en el proyecto.

3.2.4. Priorización y Consolidación

En sesiones de equipo se aplican Planning Poker (para estimar el esfuerzo) y MoSCoW (para asignar prioridad), siguiendo los lineamientos definidos en la sección 3.1.3. Una vez consensuados los resultados, se documenta la prioridad acordada y se crea un backlog inicial priorizado.

Además, se establece una revisión periódica de prioridades para ajustarlas a medida que el proyecto evoluciona. Este proceso puede repetirse para cada nueva oleada de requerimientos identificados durante el ciclo iterativo de desarrollo.

3.2.5. Resultado de análisis de requerimientos

La consolidación de cada requerimiento se registra en una plantilla estandarizada, que asegura un formato uniforme y trazable. Esta plantilla constituye la unidad de salida del modelo y el insumo directo para la conformación del backlog de desarrollo, como se muestra en la Tabla 3.

Tabla 3. Plantilla de Salida

Campo	Contenido	
Nombre del Requerimiento	(Identificador breve y representativo)	
Descripción	(Explicación clara y concisa del requerimiento)	
Tipo de Requerimiento	☐ Funcional ☐ No funcional ☐ Técnico ☐ Negocio	
Complejidad	☐ Baja ☐ Media ☐ Alta Justificación: (.....)	
Dependencias	(Módulos, requerimientos o procesos relacionados)	
Impacto en el proyecto	☐ Alto ☐ Medio ☐ Bajo	
Validación	Necesario: ☐ Sí ☐ No Apropiado: ☐ Sí ☐ No No ambiguo: ☐ Sí ☐ No Completo: ☐ Sí ☐ No Singular: ☐ Sí ☐ No Factible: ☐ Sí ☐ No Verificable: ☐ Sí ☐ No Correcto: ☐ Sí ☐ No Claro: ☐ Sí ☐ No Consistente: ☐ Sí ☐ No Viable: ☐ Sí ☐ No	
Prioridad (MoSCoW)	☐ Must Have ☐ Should Have ☐ Could Have ☐ Won't Have	
Observaciones	(Notas adicionales, decisiones tomadas, responsables involucrados)	

Esta estructura permite mantener la trazabilidad de decisiones, facilitar la comunicación entre equipos y transformar los requerimientos en historias de usuario más coherentes. En etapas futuras, se prevé que la plantilla funcione como salida estandarizada de sistemas automáticos que implementen el modelo, sirviendo de puente entre el análisis de requerimientos y la planificación técnica.

El modelo no se limita a verificar si los requerimientos cumplen con estándares teóricos de calidad, sino que promueve un proceso sistemático de mejora continua. La validación asegura que los requerimientos producidos puedan ser refinados, priorizados y transformados en historias de usuario de mayor calidad, con un impacto directo en la eficiencia de los equipos ágiles y en la calidad del software resultante.

3.3. Validación del modelo

Tras la descripción de los componentes y del proceso del modelo, resulta necesario garantizar su rigor y solidez técnica. Con este fin, la validación del marco de trabajo se realizará tomando como base el Agile Quality Framework (AQF) [26], un enfoque ya consolidado y validado para la evaluación de requerimientos ágiles.

La elección del AQF se fundamenta en tres aspectos principales:

- **Alineación conceptual:** incorpora criterios esenciales como necesidad, completitud, claridad, factibilidad y verificabilidad, que coinciden con los atributos definidos en el presente modelo.
- **Reconocimiento académico e industrial:** ha sido aplicado en experiencias empíricas y validado por la comunidad de Ingeniería de Software, lo que aporta rigor a la comparación.
- **Enfoque práctico:** sus métricas y escalas de medición resultan aplicables en entornos reales, permitiendo una evaluación sistemática que puede integrarse con herramientas automatizadas.

La validación del modelo se plantea en dos niveles complementarios:

- **Validación conceptual:** se realizó un mapeo directo entre los atributos definidos en el modelo y las características del AQF. Por ejemplo, el criterio de “no ambigüedad” del modelo se corresponde con la característica de “claridad” del AQF; la “factibilidad” técnica se alinea con el atributo de “viabilidad”; y la “consistencia” con el conjunto de requerimientos se vincula con la característica de “coherencia interna” del AQF. Este mapeo asegura que cada criterio del modelo cuenta con un respaldo normativo y metodológico reconocido.
- **Validación práctica:** en fases posteriores se prevé la aplicación del modelo en casos de estudio

controlados. Para ello se utilizarán métricas derivadas del AQF, complementadas con indicadores propios como el grado de completitud estructural, el nivel de trazabilidad y la precisión semántica de los requerimientos. Asimismo, se contempla la participación de expertos de la industria y académicos en sesiones de evaluación colaborativa, lo que permitirá contrastar la utilidad y la aplicabilidad del modelo en proyectos reales.

La complementariedad entre ambos enfoques resulta clave. Mientras que el AQF aporta definiciones estandarizadas, criterios de medición y bases teóricas consolidadas, el modelo aquí propuesto amplía esa perspectiva mediante, por un lado, un enfoque operativo que integra procesos de clasificación, validación y priorización en un flujo metodológico unificado. Y, por otro lado, la incorporación de mecanismos de validación colaborativa, que permiten considerar de manera conjunta las perspectivas de usuarios, POs y desarrolladores. Uno de estos mecanismos estará asociado a la posibilidad de automatización mediante inteligencia artificial, lo cual extiende la aplicabilidad del marco hacia escenarios de mayor escala y dinamismo.

4. Propuesta de automatización de componentes del modelo

El modelo presentado en la sección anterior constituye una primera aproximación conceptual, cuyos criterios se alinean con el Agile Quality Framework (AQF). Sin embargo, es importante destacar que aún no ha sido validado empíricamente; en esta etapa se plantean únicamente estrategias y lineamientos para su futura validación y puesta en práctica.

Para posibilitar esa evolución, y en continuidad con lo señalado en la sección 3.3 respecto de la posibilidad de automatización, se propone una arquitectura preliminar en la que la automatización ocupa un lugar central (ver Figura 1). Dicha arquitectura se organiza en torno a un *kernel* u orquestador, concebido como el componente encargado de articular de manera coherente los distintos módulos del modelo: clasificación, validación, priorización y enriquecimiento de requerimientos. El orquestador no busca reemplazar el análisis humano, sino coordinar los procesos definidos, garantizar la trazabilidad de los resultados y ofrecer una visión integrada que facilite la toma de decisiones.

El kernel cumple la función de orquestador, redirigiendo cada tarea hacia modelos de inteligencia artificial especializados, incluidos modelos generativos de propósito general como GPT¹ u otros, según el tipo de análisis requerido. De esta manera, la arquitectura se concibe como un sistema de múltiples agentes

¹ <https://openai.com/>

coordinados, donde el kernel centraliza la gestión, controla la comunicación entre módulos y asegura la coherencia de los resultados obtenidos. Esta estrategia no solo favorece la modularidad y extensibilidad del sistema, sino que también permite aprovechar las fortalezas de distintos enfoques de IA para tareas específicas, manteniendo al mismo tiempo una visión unificada del proceso.

Cada requerimiento ingresado al sistema es recibido por el kernel, que se encarga de derivarlo al módulo o modelo de IA correspondiente y de consolidar las salidas en un análisis global que incluye indicadores de calidad, observaciones contextuales y recomendaciones de mejora. El flujo no se concibe como rígido, sino como un proceso iterativo y adaptable, en el que el orquestador regula las interacciones entre los módulos y asegura que los resultados parciales se integren en una salida final coherente.

Esta propuesta arquitectónica no constituye todavía una validación del modelo, pero sí un paso metodológico necesario para preparar las condiciones en las que dicha validación podrá llevarse a cabo. En este sentido, el kernel se presenta como el elemento central de un marco integral que vincula la definición conceptual del modelo con su posible aplicación práctica, habilitando escenarios de experimentación controlada y de evolución futura hacia un agente inteligente especializado en la gestión de requerimientos ágiles.

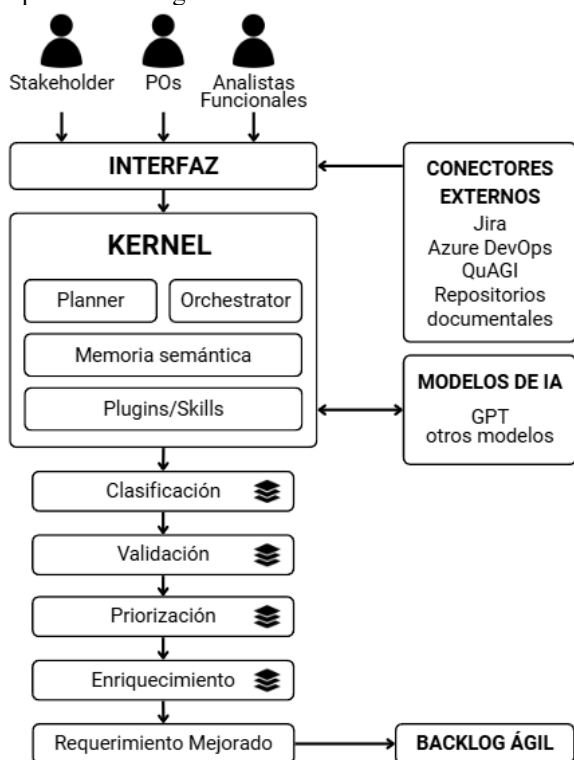


Figura 1. Arquitectura de la tecnología propuesta

4.1. Funcionalidades de la herramienta

El kernel actúa como núcleo operativo de la arquitectura, coordinando el ciclo de vida de cada requerimiento desde su ingreso hasta su consolidación final en el backlog. Bajo este esquema, el sistema no se limita a ejecutar tareas aisladas, sino que articula un conjunto de módulos especializados que permiten aplicar de forma práctica los criterios definidos en el modelo conceptual.

Entre sus principales funcionalidades se destacan: la clasificación automática de requerimientos, la validación de atributos de calidad, la asignación de prioridades y el enriquecimiento textual. Estas capacidades operan de manera integrada para garantizar que cada requerimiento sea analizado de forma sistemática y consistente.

El flujo comienza con la captura de requerimientos, ya sea mediante un formulario manual o a partir de la importación desde herramientas de gestión como Jira², Azure DevOps³ o QuAGI⁴. Una vez ingresada la información, el kernel asigna el requerimiento al módulo correspondiente, asegurando que el análisis se realice con los enfoques más adecuados según el tipo de tarea.

Los resultados parciales obtenidos en cada módulo se integran en una evaluación unificada que incluye métricas objetivas, indicadores de calidad y recomendaciones prácticas. Esta consolidación, centralizada por el kernel, proporciona a analistas, POs y equipos de desarrollo una visión integral que facilita la toma de decisiones fundamentadas.

4.2. Características técnicas

El kernel constituye el núcleo operativo de la arquitectura propuesta y es el componente que traduce el modelo conceptual definido en la Sección 3 en un proceso ejecutable dentro de la herramienta. Mientras que el modelo establece los criterios de calidad, métricas y procedimientos para evaluar requerimientos ágiles, el kernel se encarga de operacionalizarlos al coordinar los módulos que los implementan de manera automatizada. De esta forma, se establece una relación directa entre el plano teórico y el práctico: el modelo define qué debe hacerse y el kernel asegura cómo se lleva a cabo.

En primer lugar, el kernel funciona como motor de enrutamiento inteligente, aplicando los criterios definidos en el modelo conceptual para decidir qué módulo debe intervenir en cada etapa. Por ejemplo, si un requerimiento debe ser clasificado, el kernel invoca el módulo de clasificación que aplica las categorías tipológicas (funcional, no funcional, técnico, de negocio) y las dimensiones de complejidad, dependencias e impacto definidas en la Sección 3.1.1. De manera análoga, si se trata de validar atributos de calidad, el orquestador

² <https://jira.atlassian.com/>

³ <https://azure.microsoft.com/es-es/products/devops>

⁴ <http://quagi.frre.utn.edu.ar/>

redirige el requerimiento al módulo correspondiente que aplica las métricas establecidas en la Sección 3.1.2.

En segundo lugar, el kernel cumple un rol de gestor de contexto y trazabilidad. Al igual que en el modelo conceptual se subrayaba la importancia de mantener coherencia y registros consistentes, el kernel conserva los resultados intermedios de cada análisis (clasificación, validación, priorización) y los integra en una línea de tiempo trazable. Esto permite a los equipos visualizar no solo la evaluación final, sino también cómo cada criterio del modelo fue aplicado y qué modificaciones se propusieron en cada ciclo.

Asimismo, el kernel actúa como integrador de salidas heterogéneas, unificando resultados que provienen de los distintos componentes del modelo conceptual. Así, los valores numéricos derivados de métricas de validación, las etiquetas asignadas en la clasificación o las categorías de prioridad definidas en la Sección 3.1.3 convergen en un reporte consolidado. Dicho reporte reproduce la lógica de la plantilla estandarizada de salida propuesta en la Sección 3.2.5, pero generada de manera automática por el orquestador.

Un aspecto distintivo de esta arquitectura es la capacidad del kernel de funcionar como middleware de orquestación de IA. Mientras que el modelo conceptual define las dimensiones y atributos a evaluar, el kernel determina qué motor de inteligencia artificial es el más adecuado para operacionalizar cada criterio. Por ejemplo, un modelo generativo como GPT puede utilizarse para analizar la claridad semántica o reformular un requerimiento ambiguo, mientras que otros algoritmos pueden encargarse de calcular métricas de completitud o consistencia. El orquestador no solo coordina estas interacciones, sino que asegura que los resultados sean interpretados y devueltos en el marco de las categorías definidas en el modelo conceptual.

Finalmente, el kernel se diseña bajo principios de modularidad y extensibilidad, lo cual replica la flexibilidad del modelo conceptual, pensado como un marco adaptable a distintos contextos ágiles. Así, la arquitectura permite incorporar nuevos criterios de calidad, añadir métricas emergentes o integrar módulos adicionales sin alterar el flujo central. Esto refuerza la relación estrecha entre el modelo teórico y su implementación práctica, garantizando que la herramienta pueda evolucionar en paralelo con la investigación y las necesidades de la industria.

En síntesis, el kernel no es un componente aislado de soporte tecnológico, sino el vehículo que transforma el modelo conceptual en un proceso operativo. Su capacidad para enrutar tareas, preservar contexto, integrar resultados y orquestar modelos de inteligencia artificial asegura que los principios definidos en la Sección 3 puedan aplicarse de manera práctica, coherente y escalable en entornos reales de desarrollo ágil.

4.3. Proyección futura: hacia un agente inteligente especializado

La implementación de la arquitectura propuesta requiere definir un conjunto de decisiones técnicas que aseguren su viabilidad, escalabilidad y adecuación a los objetivos planteados por el modelo conceptual. En este sentido, el kernel no solo coordina los módulos, sino que también se apoya en una infraestructura tecnológica que habilita la aplicación de los criterios de calidad definidos en la Sección 3.

En primer lugar, el desarrollo del sistema demanda la selección de lenguajes y frameworks que favorezcan la modularidad y el mantenimiento a largo plazo. Tecnologías robustas para el backend, combinadas con frameworks modernos para el frontend, permitirán construir una herramienta interactiva y escalable, capaz de integrarse en los flujos de trabajo reales de los equipos ágiles. De forma complementaria, el almacenamiento de la información debe estructurarse en repositorios que registren tanto los requerimientos como sus historiales de análisis y refinamiento. Este repositorio se concibe como un componente estratégico, ya que no solo soporta la trazabilidad de las evaluaciones, sino que también constituye la base para procesos de aprendizaje incremental en los modelos de IA involucrados.

Un segundo aspecto técnico central es la incorporación de técnicas de procesamiento de lenguaje natural (NLP). Estas técnicas se utilizarán para operacionalizar los atributos de calidad definidos en el modelo conceptual, en especial aquellos vinculados con la claridad, la no ambigüedad y la completitud de los requerimientos. El uso de NLP permitirá analizar en tiempo real el texto de los requerimientos, detectar inconsistencias y generar reformulaciones o sugerencias que mejoren la calidad del enunciado.

En tercer lugar, la interoperabilidad constituye un requisito indispensable. El kernel debe habilitar la integración con herramientas de gestión de proyectos como Jira, Azure DevOps o QuAGI, de manera que los equipos puedan beneficiarse de las funcionalidades de la arquitectura sin modificar drásticamente sus entornos de trabajo. Para ello, se plantea el uso de APIs estandarizadas que faciliten el intercambio de información y la sincronización de requerimientos entre sistemas.

Desde la perspectiva de proyección futura, esta arquitectura preliminar abre el camino hacia la construcción de un agente inteligente especializado en análisis de requerimientos. Dicho agente podría evolucionar a partir de la infraestructura actual, incorporando capacidades de aprendizaje continuo y adaptación a contextos particulares de cada organización o proyecto. En este escenario, el kernel se erigiría como mediador entre los criterios del modelo conceptual y las decisiones autónomas del agente, garantizando que la

evolución tecnológica no implique una desvinculación de los principios teóricos que sustentan la propuesta.

La combinación de modularidad, trazabilidad, procesamiento de lenguaje natural e interoperabilidad configura una base tecnológica alineada con los principios del modelo conceptual. Esta infraestructura preliminar sienta las condiciones necesarias para su implementación práctica y, al mismo tiempo, prepara el terreno para futuras instancias de validación empírica y evolución hacia sistemas más autónomos de apoyo a la gestión de requerimientos.

5. Conclusiones y trabajos futuros

El trabajo presentado constituye una primera aproximación a un marco de calidad para la gestión de requerimientos en entornos ágiles, estructurado en torno a tres componentes principales: clasificación, validación y priorización. La propuesta avanza respecto de antecedentes como el Componente N° 4 del modelo QuAM al integrar no solo criterios de calidad conceptuales, sino también métricas operativas y una plantilla de salida que sistematiza los resultados. De este modo, se ofrece una estructura que combina atributos cuantificables con lineamientos prácticos para la evaluación y mejora de los requerimientos desde etapas tempranas del desarrollo.

El modelo contribuye a la literatura de Ingeniería de Software Empírica al plantear un flujo metodológico integral que articula tanto la evidencia previa —derivada de revisiones sistemáticas y encuestas— como mecanismos de medición que permiten establecer índices de calidad observables y reproducibles. La propuesta de incorporar un kernel como orquestador de los distintos módulos abre, además, un camino hacia la automatización y la interoperabilidad con herramientas de uso extendido en la industria, lo cual refuerza la aplicabilidad práctica del enfoque.

No obstante, esta investigación se encuentra aún en una fase conceptual y requiere de instancias posteriores de validación. Entre los trabajos futuros se destacan: (i) la aplicación del modelo en estudios de caso controlados que permitan verificar su pertinencia y robustez en contextos reales; (ii) el desarrollo de prototipos de la herramienta que operacionalicen el kernel y los módulos de clasificación, validación y priorización; (iii) la incorporación de técnicas de procesamiento de lenguaje natural para evaluar atributos como claridad, completitud y consistencia; y (iv) la evolución hacia un agente inteligente especializado en gestión de requerimientos, capaz de aprender de interacciones previas y adaptarse a contextos particulares de proyecto.

En síntesis, el marco propuesto constituye un paso inicial hacia la mejora sistemática de los procesos de

gestión de requerimientos en entornos ágiles. Su consolidación a través de validaciones empíricas y desarrollos tecnológicos permitirá evaluar de manera más precisa cómo la calidad de los requerimientos incide en la calidad tanto del proceso como del producto de software, contribuyendo a cerrar la brecha entre investigación académica y práctica industrial.

Agradecimientos

Este trabajo se enmarca en las actividades relacionadas con el Proyecto de Investigación [REDACTED]

Agradecemos el invaluable aporte de los docentes investigadores, miembros del proyecto nombrado anteriormente, [REDACTED].

Referencias

- [1] Gupta, A., Poels, G., & Bera, P., “Using Conceptual Models in Agile Software Development: A Possible Solution to Requirements Engineering Challenges in Agile Projects”, IEEE Access, 10, 2022, pp. 119745-119766.
- [2] Franch, X., Palomares, C., Quer, C., Chatzipetrou, P., & Gorshek, T., “The state-of-practice in requirements specification: an extended interview study at 12 companies”, Requirements Engineering, 28(3), 2023, pp. 377-409.
- [3] digital.ai, “17th State of Agile Report”, 2024, <https://info.digital.ai/rs/981-LQX-968/images/RE-SA-17th-Annual-State-Of-Agile-Report.pdf>
- [4] Meckenstock, J. N., “Shedding light on the dark side – A systematic literature review of the issues in agile software development methodology use”, Journal of Systems and Software, 211, 2024, p. 111966.
- [5] Muhammad, A. P., Knauss, E., Batsaikhan, O., Haskouri, N. E., Lin, Y. C., & Knauss, A., “Defining Requirements Strategies in Agile: A Design Science Research Study”, en International Conference on Product-Focused Software Process Improvement (PROFES 2022), Jyväskylä, Finland, 21 al 23 de noviembre de 2022, pp. 73-89.
- [6] Zelkowitz, M., & Wallace, D., “Experimental validation in software engineering”, Information and Software Technology, 39(11), 1997, pp. 735-743.
- [7] Basili, V., Shull, F., & Lanubile, F., “Building knowledge through families of experiments”, IEEE Transactions on Software Engineering, 25(4), 1999, pp. 456-473.

- [8] Tomaselli, G. P., Pinto, N. S., & Acuña, C., "Requirements management methods and practices for improving the quality of agile software development processes: a review of the literature", *Requirements Engineering*, 2025. <https://doi.org/10.1007/s00766-025-00448-3>
- [9] Tomaselli, G., Pinto, N., Acuña, C., Martínez, D., & Ferrazzano, A., "Aplicación de Técnicas de Ingeniería de Software Empírica para Relevar Prácticas Actuales en la Gestión de Requerimientos Ágiles", en *Memorias 12vo. Congreso Nacional de Ingeniería Informática y Sistemas de Información (CoNaIISI 2024)*, Catamarca, 7 y 8 de noviembre de 2024, pp. 627-637.
- [10] Pressman, R., & Maxim, B. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education, New York, USA.
- [11] Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education, Harlow, UK.
- [12] ISO/IEC/IEEE. (2018). ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering. International Organization for Standardization (ISO), International Electrotechnical Commission (IEC) and Institute of Electrical and Electronics Engineers (IEEE), Geneva, Switzerland. Disponible en: <https://www.iso.org/standard/72089.html>
- [13] Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., ... Thomas, D. (2001). *Manifesto for Agile Software Development*. Disponible en: <https://agilemanifesto.org>
- [14] Cohn, M. (2004). *User Stories Applied: For Agile Software Development*. Addison-Wesley Professional, Boston, MA, USA.
- [15] Boehm, B., & Basili, V. R. (2005). Software Defect Reduction Top 10 List. In V. R. Basili, C. R. V. Oliveira, F. Q. da Silva, A. C. C. de Melo, & J. C. Carver (Eds.), *Foundations of Empirical Software Engineering: The Legacy of Victor R. Basili* (pp. 426–431). Springer, Berlin, Germany.
- [16] Wiegers, K., & Beatty, J. (2013). *Software Requirements* (3rd ed.). Microsoft Press, Redmond, WA, USA.
- [17] Kotonya, G., & Sommerville, I. (1998). *Requirements Engineering: Processes and Techniques*. Wiley Publishing, Chichester, UK.
- [18] ISO/IEC. (2023). ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model. International Organization for Standardization (ISO), Geneva, Switzerland. Disponible en: <https://www.iso.org/standard/78176.html>
- [19] CMMI Institute. (2023). CMMI® Version 3.0 – Capability Maturity Model Integration: A Process Improvement Framework. Pittsburgh, PA: CMMI Institute. Disponible en: <https://cmmiinstitute.com/>
- [20] Pinto N, Acuña C, Cuenca Pletsch LR (2016) Quality Evaluation in Agile Process: A First Approach. In: XXII Congreso Argentino de Ciencias de la Computación (CACIC 2016), pp. 525-534.
- [21] Pinto N, Tomaselli G, Acuña C, Cuenca Pletsch L (2017) QuAGI: Una propuesta para el seguimiento y evaluación de proyectos de Software Ágiles. In: V Seminário Argentina-Brasil de Tecnologias da Informação e da Comunicação (SABTIC 2017), pp 127-136. SETREM - Sociedade Educacional Três de Maio. <https://doi.org/10.5281/zenodo.583174>
- [22] IEEE. (1998). IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications. Institute of Electrical and Electronics Engineers, New York, USA. Disponible en: <https://doi.org/10.1109/IEEESTD.1998.88286>
- [23] Clegg, D., & Barker, R. (1994). *Case Method Fast-Track: A Rad Approach*. Addison-Wesley Longman Publishing Co., Inc., USA.
- [24] Waters, K. (2009). Prioritization using MoSCoW. *Agile Planning*, 12(31).
- [25] Cohn, M. (2005). *Agile Estimating and Planning*. Prentice Hall, Upper Saddle River, NJ, USA.
- [26] Pinto N, Tortosa N, Geat BC, Ibáñez L, Bollati V (2018) Quality evaluation of agile processes: Measurement of requirements management using AQF v2. In: 11th International Conference on the Quality of Information and Communications Technology (QUATIC 2018), pp 15-20. IEEE.